

Reproduceerbaarheid testen van een R-gebaseerd bioinformatica-project in JupyterLab

Jalisa Shaline van der Zeeuw

2026-01-01

In dit bewijsstuk laat ik zien hoe ik de reproduceerbaarheid van een project van mijn collega heb gecontroleerd (criterium 12, samenwerken en communiceren met betrokkenen). Wij zijn samen in de vakantie naar de HU gegaan om onze reproduceerbaarheid te controleren. Aan het einde van de dag heb ik mijn collega feedback gegeven met tips die haar uiteindelijk hebben geholpen.

Mijn collega had haar analyse uitgevoerd in Rstudio en beschikbaar gemaakt via Github. Om te onderzoeken of haar werk ook op mijn eigen systeem uitvoerbaar is, heb ik het project lokaal gekloond en geprobeerd de RMarkdown-analyse uit te voeren binnen JupyterLab in plaats van RStudio. Hier heb ik bewust voor gekozen om te laten zien dat ik eigenaarschap toon en bereid ben mij extra in te zetten (criterium 10). Ik heb onderzocht hoe ik een R-kernel kan instellen en mijn werkomgeving geschikt maak voor het uitvoeren van andermans code.

Daarnaast toon ik aan dat ik veilig en bewust werk in mijn digitale omgeving (criterium 3). Ik ga zorgvuldig om met de code van een collega door de repository gecontroleerd te klonen. Ik bekijk eerst de inhoud, voer de code niet blindelings uit en werk binnen een geïsoleerde omgeving (R-kernel in JupyterLab).

Projectmap aanmaken

Voor het controleren van de reproduceerbaarheid van het project van mijn collega Noa van Dam (**lncRNA_sc_splice**) heb ik eerst een duidelijke mappenstructuur aangemaakt. Ik heb een algemene map **project_collegas** gemaakt, waarin ik voor elke collega een eigen submap kan aanmaken. Op deze manier houd ik overzicht over de verschillende projecten die ik eventueel wil controleren.

In dit geval heb ik de submap **noa** aangemaakt, waarna ik het werkp pad heb ingesteld naar deze locatie.

Met `getwd()` controleer ik of ik me in de juiste directory bevind.

Deze stap draagt bij aan bewust en georganiseerd werken, omdat ik hiermee een veilige, overzichtelijke en reproduceerbare omgeving creëer.

```
# maak een map aan genaamd project_collegas
dir.create("/Users/jalisavanderzeeuw/project_collegas/noa", showWarnings = FALSE, recursive = TRUE)

# ga naar de juiste map
setwd("/Users/jalisavanderzeeuw/project_collegas/noa")

# controleer of je in de goede map zit
getwd()
```

```
‘/Users/jalisavanderzeeuw/project_collegas/noa’
```

Repository klonen en inhoud bekijken

Het project van mijn collega is beschikbaar via GitHub. Door gebruik te maken van een HTTPS-link kan de repository lokaal worden gekloond. Ik gebruik een if-else-structuur om te controleren of de map `lncRNA_sc_splice` al bestaat. Dit houdt in dat als de map nog niet bestaat, de repository wordt gekloond. Als deze map al wel aanwezig is, wordt klonen overgeslagen om dubbele bestanden of conflicten te voorkomen.

Daarna bekijk ik de inhoud van de repository. Dit helpt om inzicht te krijgen en om te controleren of er belangrijke documenten aanwezig zijn, zoals een README-bestand. Een README kan worden gezien als een handleiding voor het project. Hierin hoort te staan hoe je het project kunt uitvoeren. In dit geval bleek het README-bestand nog leeg te zijn. Dit heb ik genoteerd als feedbackpunt richting mijn collega.

Deze stap toont samenwerking doordat ik feedback geef over de documentatie en de reproduceerbaarheid van haar werk.

```
# clone GitHub repo als de map nog niet bestaat
if(!dir.exists("lncRNA_sc_splice")) {
  message("Repository wordt gekloond...")
  system("git clone https://github.com/ProjecticumDlerpDs/lncRNA_sc_splice.git")
} else {
  message("Repository bestaat al, wordt niet opnieuw gekloond.")
}

# inhoud repo bekijken
list.files("lncRNA_sc_splice", recursive = TRUE)
```

1. ‘lncRNA_sc_splice.Rproj’
2. ‘raw_data/hg19/barcodes.tsv’
3. ‘raw_data/hg19/genes.tsv’
4. ‘raw_data/hg19/matrix.mtx’
5. ‘README’
6. ‘seurat/Data_8_5_analyse.Rmd’
7. ‘seurat/seurat_tutorial.html’
8. ‘seurat/seurat_tutorial.Rmd’

Pad naar Rmarkdown-bestand controleren

In deze stap heb ik het pad ingesteld naar het RMarkdown-bestand dat ik wilde uitvoeren: `seurat_tutorial.Rmd`. Ik controleer of het bestand aanwezig is op het opgegeven pad. Dit heb ik gedaan met de if-structuur zodat wanneer dit niet het geval is dit wordt weergegeven.

Deze stap laat zien dat ik bewust en zorgvuldig werk, omdat ik controleer of alle benodigde bestanden correct beschikbaar zijn voordat ik verder ga.

```
# pad naar Rmd
rmd_path <- "/Users/jalisavanderzeeuw/project_collegas/nea/lncRNA_sc_splice/seurat/seurat_tutorial.Rmd"

# check of het bestand bestaat
if(!file.exists(rmd_path)) stop("Rmd-bestand niet gevonden!")
```

Rmarkdown runnen

Vervolgens heb ik geprobeerd om het RMarkdown-bestand te runnen met de functie `rmarkdown::render()`. Hiermee wilde ik de analyse van mijn collega opnieuw uitvoeren en controleren of de resultaten reproduceerbaar zijn.

Opmerking! De codecel hieronder geef ik weer als opmerking omdat de foutmelding die volgt enorm lang is

```
rmarkdown::render(
  input = rmd_path,
  output_format = "pdf_document", # of "html_document"
  output_dir = "/Users/jalisavanderzeeuw/project_collegas/nea",
  quiet = TRUE
)
```

```
# Rmd renderen
# rmarkdown::render(
#   # input = rmd_path,
#   # output_format = "pdf_document", # kan ook html_document
#   # output_dir = "/Users/jalisavanderzeeuw/project_collegas/nea",
#   # quiet = FALSE
# )
```

Tijdens het uitvoeren kreeg ik een hele lange foutmelding. De eerste foutmelding:

```
`Error in Read10X(): Directory provided does not exist`
```

Deze melding geeft aan dat het pad naar de dataset die in de RMarkdown-code wordt ingelezen niet bestaat op mijn computer. De oorzaak ligt dus bij de manier waarop de paden in het project zijn vastgelegd. Deze verwijzen namelijk naar een map op haar computer of de server waarop ze werkt.

Ik heb dit opgenomen in mijn feedback, met het advies om gebruik te maken van relatieve paden. Als tip heb ik gegeven om te verdiepen in het `here`-pakket of `file.path()`.

Extra onderdeel > R-kernel instellen in JupyterLab

Omdat het project van mijn collega in RStudio is ontwikkeld, moest ik mijn eigen werkomgeving (JupyterLab) geschikt maken voor het uitvoeren van R-code. Hiervoor heb ik een R-kernel toegevoegd aan JupyterLab. Een kernel fungeert als de rekenomgeving waarin code wordt uitgevoerd en bepaald welke programmeertaal binnen een notebook wordt gebruikt.

Het toevoegen van de R-kernel bestond uit het installeren van het R-pakket `IRkernel` en het registreren van de kernel met het commando `IRkernel::installspec(user = TRUE)`. Hierdoor kon ik binnen JupyterLab direct werken in een R-omgeving, zonder te hoeven wisselen van software.

Dit proces heeft mij een aantal uren gekost, terwijl ik het controleren op reproduceerbaarheid ook gewoon in RStudio had kunnen uitvoeren. Ondanks dat beschouw ik deze stap als leerzaam en heb ik hiermee meer inzicht gekregen in de werking van Jupyter en in de manier waarop verschillende programmeertalen binnen één omgeving geïntegreerd kunnen worden. Ik ben nu in staat om flexibeler te werken met verschillende soorten kernels en programmeertalen.